CSE-2340 · SOFTWARE DEVELOPMENT 1

Week 6 · Classes 3 and 4

Events

addEventListener, the event object, preventDefault, and a small interactive app

Md Sadman Hafiz

Lecturer, Dept. of CSE, IIUC

Autumn 2026

C3 · 0-5 MIN

Recall and today's outcome

RECALL

From Classes 1 and 2

- What are the three steps for making an element?
- What does querySelector return when nothing matches?
- Why was bug 3 reported twenty lines from the real problem?
- Who finished renderBooks(list)?

OUTCOME

By 50 minutes you can

- Run your code when a person does something
- Stop a form reloading the page
- Read what someone typed into an input
- Add and remove items and see the page follow

Today the restriction lifts

- C1 and C2 banned addEventListener on purpose, so that building elements became automatic first.
- That ban is over. Everything you built on Monday is about to start responding.

If you wrote renderBooks(list) for the home task, today is one new line for you. If you did not, write it in the first ten minutes.

C3 · 5-15 MIN · CONCEPT

Your code stops. Then it waits.

Page loads

Your script runs top to bottom, once

→

Script ends

Nothing is running any more

→

A person acts

Click, type, submit, scroll

→

The browser calls you

It runs the function you left behind

This is the big idea

- You do not run your function
- You hand it to the browser and walk away
- The browser runs it, later, if it needs to

It may run many times

- Or never, if nobody clicks
- Each run gets fresh information about what happened

The name for this

A function you give to someone else to call is a callback. You already wrote them: map, filter and find all take one.

addEventListener: three parts

```
const button = document.querySelector("#add-btn");

button.addEventListener("click", handleAdd);
//     ^target        ^type     ^the function to call
```

TARGET

button

Which element is being watched

TYPE

"click"

What to watch for, as a string

HANDLER

handleAdd

The function the browser will call

```
// Three ways to write the handler. All identical in effect.
button.addEventListener("click", handleAdd);                  // named, defined elsewhere
button.addEventListener("click", function (e) { ... });       // inline
button.addEventListener("click", (e) => { ... });             // arrow, the usual choice
```

The parentheses trap

```
// WRONG
button.addEventListener(
  "click",
  handleAdd()
);
```

```
// RIGHT
button.addEventListener(
  "click",
  handleAdd
);
```

What the wrong one does

- Runs handleAdd immediately, as the page loads
- Hands the browser whatever it returned, usually undefined
- Clicking then does nothing, forever

The rule

- () means run it now
- No () means here is the function, you run it
- You are passing a recipe, not a cooked meal

How you will recognise it

Something happens once when the page loads, and then clicking is dead. That is always this bug.

The event object

```
button.addEventListener("click", (e) => {
  console.log(e.type);           // "click"
  console.log(e.target);         // what was actually clicked
  console.log(e.currentTarget);  // what the listener is on
  e.preventDefault();            // stop the browser's default
});
```

The browser passes it in

You never create it. Just accept it as the parameter.

Call it e

e, evt or event. Any name works. e is the convention.

target and currentTarget differ

- Listener on a card, person clicks the title inside it
- e.target is the title. e.currentTarget is the card
- You almost always want currentTarget

The symptom when you get this wrong

It works when you click the edge of a card and does nothing when you click the words. Same click, different target.

The events you will actually use

Type	Usually on	Fires when
`click`	button, link, anything	A person clicked it
`submit`	form	The form is being sent. Needs preventDefault
`input`	input, textarea	Fires on every keystroke
`change`	select, checkbox	Fires when the value is settled
`keydown`	input, document	A key went down. e.key tells you which
`mouseover`	anything	The pointer entered it

The three in green cover almost everything in this course. Learn those properly and look the rest up when you need them.

preventDefault: the one that saves forms

A form's default is to send itself and reload the page

Reload means a fresh DOM. Everything your script built is gone. The list empties. It looks like your code did nothing.

```
const form = document.querySelector("#book-form");

form.addEventListener("submit", (e) => {
  e.preventDefault();      // <-- without this, the page reloads

  const title = titleInput.value;
  books.push({ title });
  renderBooks(books);
  form.reset();
});
```

Listen on the form, not the button

- submit on the form also catches the Enter key
- click on the button misses it

Remember Class 1

- The DOM is memory, not your file
- A reload rebuilds it from the HTML
- That is why the list empties

Also has a default

- A link navigates away
- A checkbox ticks itself
- Right-click opens a menu

Reading what someone typed

```
const titleInput = document.querySelector("#title");

// READ IT INSIDE THE HANDLER, every time it runs
form.addEventListener("submit", (e) => {
  e.preventDefault();

  const title = titleInput.value.trim();
  const year  = Number(yearInput.value);

  if (title === "") return;      // nothing typed, stop here

  books.push({ title, year });
  renderBooks(books);
  form.reset();
});
```

Always guard the empty case

- .trim() removes spaces, so " " counts as empty
- An early return is the simplest guard there is

TOP BUG

Reading it too early

- const title = titleInput.value at the top of the file
- That runs once, at load, when the box is empty
- You then add empty items forever

value is always a string

- Even from a number input
- Wrap it: Number(input.value)

form.reset()

Clears every field in one call

The shape of every interactive app

1 Something happens

```
A person clicks or types
```

→

2 Change the data

```
books.push(...) or
books.filter(...)
```

→

3 Redraw from the data

```
renderBooks(books)
```

The array is the truth. The page is a picture of it.

Never edit the page and the data separately. Change the array, then redraw. The page can never disagree with itself.

The tempting shortcut

- Deleting a card with element.remove()
- It vanishes, so it looks finished
- It comes back on the next redraw

Why it comes back

- You removed the picture, not the thing
- The array still has it

Listeners on elements you created

```
function renderBooks(list) {
  library.innerHTML = "";                    // clear first, always

  for (const book of list) {
    const article = document.createElement("article");
    const btn = document.createElement("button");
    btn.textContent = "Remove";

    // attach the listener as you create the button
    btn.addEventListener("click", () => {
      removeBook(book.id);
    });

    article.appendChild(btn);
    library.appendChild(article);
  }
}
```

WHY NOT OUTSIDE

A listener needs an element

- Attach it at load and the buttons do not exist yet
- querySelectorAll finds nothing, silently

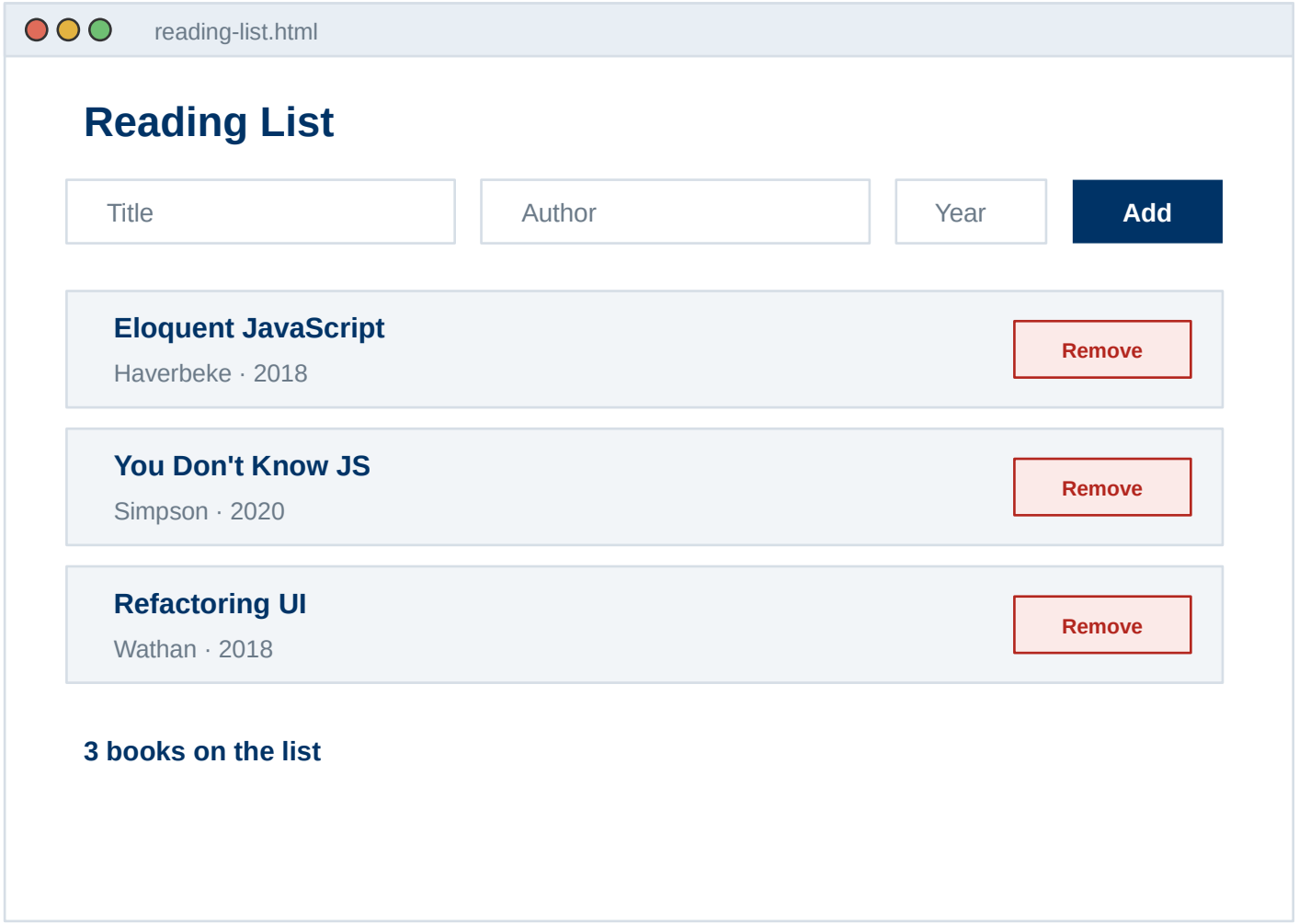
Clear before you draw

Forget innerHTML = "" and every render doubles the list.

Later: delegation

One listener on the parent. C4 Exercise 3.

Live build: a reading list



What it does

- Type a book and press Add, or press Enter
- It appears in the list immediately
- Remove takes it out again
- The count follows on its own

Everything is from this week

- createElement and appendChild from Monday
- addEventListener and preventDefault from today
- One array, one render function

Steps 1 and 2: render, then add

STEP 1 — THE RENDER FUNCTION

```
function renderBooks(list) {
  shelf.innerHTML = "";          // clear
  for (const b of list) {
    const row = document.createElement("li");
    row.textContent = `${b.title} — ${b.year}`;
    shelf.appendChild(row);
  }
  count.textContent =
    `${list.length} books on the list`;
}

renderBooks(books);              // draw once at load
```

STEP 2 — THE FORM

```
form.addEventListener("submit", (e) => {
  e.preventDefault();

  const title = titleInput.value.trim();
  if (title === "") return;

  books.push({
    id: Date.now(),
    title,
    year: Number(yearInput.value)
  });

  renderBooks(books);
  form.reset();
});
```

Test step 1 before step 2

Three books and the count must appear on load, with no form at all.

Why Date.now() as the id

Every book needs something unique so Remove knows which one to take out.

Step 3: remove

```
function renderBooks(list) {
  shelf.innerHTML = "";

  for (const b of list) {
    const row = document.createElement("li");
    row.textContent = `${b.title} — ${b.year}`;

    const btn = document.createElement("button");
    btn.textContent = "Remove";
    btn.addEventListener("click", () => {
      books = books.filter(x => x.id !== b.id);   // change the data
      renderBooks(books);                         // redraw
    });

    row.appendChild(btn);
    shelf.appendChild(row);
  }
}
```

NOT THIS

row.remove()

- The card disappears, so it looks done
- The array still has the book
- Add anything and it comes straight back

books must be let

filter returns a NEW array, so you reassign.

Change the data, then redraw. Every time.

Mistakes you will make today

It ran on load, now clicking is dead

You wrote handleAdd() with parentheses.

The page flashes and the list empties

No e.preventDefault() in the submit handler.

It adds empty items

You read input.value at load instead of inside the handler.

The list doubles every time

You forgot innerHTML = "" at the top of render.

Deleted items come back

You removed the element, not the item in the array.

Works on the edge, not on the words

e.target is the inner element. Use e.currentTarget.

C3 · 48-50 MIN

Exit check

CONCEPTUAL

Answer in one sentence

- Why does a form reload the page, and why does that empty your list?
- What is the difference between handleAdd and handleAdd()?

PRACTICAL

Show me

- Add a book with the Enter key
- Remove one and watch the count follow

The sentence to take away

The array is the truth. The page is a picture of it. Change the data, then redraw.

Next class

C4 is practice: you extend this app, then debug six planted bugs. Bring your reading-list files.

WEEK 6 · CLASS 4

Practice: a small interactive application

Extend it, then find six bugs that produce no error at all

C4 · 0-5 MIN

The plan for this hour

1	**Finish and polish**	5-14 min	Guard empty input, clear the form, keep the count honest
2	**Add a filter**	14-24 min	An input event that narrows the list without touching the data
3	**One listener, not many**	24-34 min	Event delegation on the parent, with e.target.closest
4	**Six planted bugs**	34-46 min	Five are silent. The app works, just not correctly

Last 4 minutes: assignment Part 1 check. Exercise 3 is the one that matters for Mini Project 3.

C4 · 5-24 MIN

Exercises 1 and 2

EXERCISE 1

Finish and polish

- Refuse an empty title, and say so on the page
- Clear the form after a successful add
- Show "No books yet" when the list is empty
- Make the count read "1 book" and "3 books" correctly

EXERCISE 2

A live filter

- A search box above the list
- On input, show only matching titles
- Do NOT delete anything from the array
- Clearing the box brings everything back

```
searchInput.addEventListener("input", (e) => {
  const term = e.target.value.toLowerCase();
  const shown = books.filter(b => b.title.toLowerCase().includes(term));
  renderBooks(shown);        // draw a SUBSET. books itself is untouched.
});
```

This is why renderBooks takes a list instead of reading books directly. Now you can draw any list you like.

Exercise 3: one listener instead of many

```
// Instead of a listener per button, one on the parent:

shelf.addEventListener("click", (e) => {
  const btn = e.target.closest(".remove-btn");
  if (!btn) return;                // they clicked something else

  const id = Number(btn.dataset.id);
  books = books.filter(b => b.id !== id);
  renderBooks(books);
});

// and in render, instead of addEventListener:
btn.classList.add("remove-btn");
btn.dataset.id = b.id;
```

This is the Mini Project 3 pattern

- Any list where items are added, removed or edited
- Worth getting comfortable with today, not in Week 8

Why this is better

- One listener, written once
- It keeps working for buttons that do not exist yet
- Nothing to clean up when you redraw

closest is the trick

- e.target may be text inside the button
- closest walks up until it finds a match
- Returns null if there is none, so guard it

C4 · 34-46 MIN

Exercise 4: six planted bugs

The app runs. It just behaves wrongly.

Only one of these six produces a Console error. For the rest, you have to notice that the behaviour is wrong.

1	An error at load, and Add does nothing	Console
2	The list explodes: 3, then 7, then 12 rows	Silent
3	Added books have no title	Silent
4	Removed books come back	Silent
5	The search box does nothing at all	Silent
6	Searching destroys your books for good	Silent

Watch the count and the page disagree

- While bug 2 is alive the count says 4 and the page shows 7 rows. The count was right the whole time
- The count reads the data. The data was never wrong. Only the picture was

C4 · 46-50 MIN

Assignment (5 marks)

PART 1 · 2 MARKS

In class, last 4 minutes

- Add a book with the Enter key, then remove it
- Show the filter narrowing the list without losing data
- Say why the array is the truth and the page is a picture

PART 2 · 3 MARKS

Home, before Week 7 C1

- Fix all six planted bugs
- Convert Remove to event delegation
- Add an Edit button that puts a book back in the form
- Commit and push to your Week 6 repository

Rubric

- Add, remove and count all work from the data — 1.5 · Filter narrows without destroying the array — 1.0
- All six bugs fixed — 1.5 · Delegation working and pushed — 1.0

Week 7 is the mid-term week

C3 and C4 next week are the 100-minute practical, 20 marks. It needs semantic HTML, responsive CSS and one real interaction. This app is the rehearsal.

C4 · 48-50 MIN

Exit check and what comes next

CONCEPTUAL

Answer in one sentence

- Why does removing the element not remove the book?
- Why does one listener on the parent still work for new buttons?

PRACTICAL

Show me

- Two adds and a remove, with the count correct
- The filter box emptied, restoring every book

Week 7

- C1 and C2: array methods for UI work, and preparation for Mini Project 3
- C3 and C4: the mid-term practical examination, 100 minutes, 20 marks

You now have every piece of a working application: data, a render function, and events. The rest of this course is that loop, made larger.